1.调用skill生成需求描述文档的情况：

```
Claude Code v2.1.71

Welcome back!

Sonnet 4.6 · API Usage Billing
~\Desktop\CS2026\AISE\rag_backend

Tips for getting started
Run /init to create a CLAUDE.md file with instructions for Claude

Recent activity
No recent activity

> 你是一个软件工程师，现在请你根据需求描述文档：项目3.md 文件中的描述，写一版软件需求规格说明书（最后说明你使用了哪些skills和mcp服务）

● Read 1 file (ctrl+o to expand)

● 好，已读取需求文档，现在调用 se-requirements-doc-assistant skill 来生成软件需求规格说明书。

● Skill(se-requirements-doc-assistant)
  └ Successfully loaded skill

● 好，先读取模板文件，再生成完整的需求规格说明书。

● Read 1 file (ctrl+o to expand)

● 模板已读取。现在生成完整的软件需求规格说明书并写入文件。

● Write(~\Desktop\CS2026\AISE\软件需求规格说明书_饱了么.md)
  └ Wrote 983 lines to 软件需求规格说明书_饱了么.md
    # 软件需求规格说明书

    **项目名称**：饱了么 校园外卖下单平台
    **开发单位**：饕餮科技有限公司
    **文档版本**：v1.0
    **编写日期**：2026-03-08
    **文档状态**：草稿

    ---

    … +973 lines (ctrl+o to expand)
```

产出文件

软件需求规格说明书_饱了么.md — 已写入项目根目录

文档结构概览

章节	内容
第1章 范围	系统标识、概述、文档说明、基线
第2章 引用文件	需求来源文档与参考系统
第3章 需求（核心）	共24小节，覆盖全部需求维度
├─ 3.2 需求概述	目标、运行环境、用户特点、关键点、约束
├─ 3.3 需求规格	系统总体结构、ER概览、描述约定
├─ 3.4 CSCI能力需求	**91条 FR 功能需求**，按用户点餐/商家接单/骑手配送/管理员/增值模块分组，每条含触发条件、处理规则、验收标准、状态标注
├─ 3.5 外部接口需求	6个外部接口（移动端、地图API、支付、推送等）
├─ 3.7 内部数据需求	4个核心实体字段定义 + 一致性策略
├─ 3.10 安全与隐私需求	加密、脱敏、权限隔离（RBAC）
├─ 3.13 软件质量因素	性能(P95)、可靠性(99.5%)、易用性、可扩展性
├─ 3.18 算法说明	智能派单评分公式、推荐算法概述
├─ 3.24 需求优先次序	P0/P1/P2/P3 四档优先级划分
第4章 合格性规定	15+ 条需求的验收方法（演示/测试/分析/审查）
第5章 需求追踪矩阵	需求编号 → 设计模块 → 测试用例编号
第6章 尚未解决的问题	**10条 [待确认] 问题清单**，含优先级
第7章 注解	术语表、缩略词表
附录A/B	用例图（文本版）、范围内外说明

使用的 Skills 与 MCP 服务

Skills 使用情况

Skill	作用
se-requirements-doc-assistant	核心 skill，提供需求文档工作流（分层提炼、接口/数据梳理、追踪链建立、待确认问题管理）和模板资产（assets/软件需求规格说明书.md）。本文档严格遵循该模板的章节结构，不删除任何模板条目，对不适用项进行了说明。

MCP 服务使用情况

本次任务**未调用 MCP 服务**。所有内容均来自：
1. 读取项目本地文件（项目3.md、skill 模板 软件需求规格说明书.md）
2. Claude 模型基于需求描述和软件工程知识进行结构化生成

如需进一步调用 RAG 后端检索相关参考资料（如项目已有架构文档），可通过 rag_backend 服务增强内容质量。

✻ Churned for 5m 40s

从提问到生成文档共计5m40s，本过程不需要调用rag数据库查询。时间长感觉主要是因为api key反映比较慢

2. 调用rag库但没有收索到结果的情况

```
❯ 根据已知文档（需求描述和之前生成的软件需求规格说明书），写一版软件概要说明书说明书，如果有细节不清楚的地方，可以参考rag库中的知识（最后说明你使用了哪些skills和mcp服务）

● Searched for 1 pattern, read 4 files (ctrl+o to expand)

● MCP 服务和模板都已读取。现在先用 RAG 检索关键章节参考内容，同时触发 skill。

● Skill(se-architecture-doc-assistant)
  └ Successfully loaded skill

● 好，先并行查询 RAG 库中的设计参考，同时读取已有的需求规格说明书。

● Read 1 file (ctrl+o to expand)

● Bash(cd "C:\Users\chuanzhenqi\Desktop\CS2026\AISE\rag_backend" && python -c "
      import sys…)
  └ /usr/bin/bash: line 22: python: command not found

● Bash(cd "C:\Users\chuanzhenqi\Desktop\CS2026\AISE\rag_backend" && python3 -c "
      import sys…)
  └ Error: Exit code 127
    /usr/bin/bash: line 22: python3: command not found

● Bash(cd "C:\Users\chuanzhenqi\Desktop\CS2026\AISE\rag_backend" && myenv/Scripts/python.exe -c "
      import sys…)
  └ 2026-03-08 18:14:27.623 | INFO     | vectordb.retriever:search:377 - ►►►►►►►query='cT►►►►►►►
```

```
● RAG 库目前向量集合为空（0 条命中），已确认无历史文档可参考。直接基于 SRS 文档生成概要设计说明书。

● Read 1 file (ctrl+o to expand)

● 信息已足够，开始生成完整的软件概要设计说明书。

● Write(~\Desktop\CS2026\AISE\软件概要设计说明书_饱了么.md)
  └ Wrote 857 lines to ..\软件概要设计说明书_饱了么.md
    # 软件概要设计说明书

    **项目名称**: 饱了么 校园外卖下单平台
    **开发单位**: 饕餮科技有限公司
    **文档版本**: v1.0
    **编写日期**: 2026-03-08
    **文档状态**: 草稿

    ---

    … +847 lines (ctrl+o to expand)

● 文档已生成完毕，保存至 软件概要设计说明书_饱了么.md。

  ---
  文档结构概览
```

章节	内容
1 引言	编写目的、背景（任务提出方/开发方/用户/运行站点）、完整术语表（19条）、参考文件
2 系统需求概述	四角色输入输出规定、5大业务目标、运行环境（设备+支持软件清单）、设计约束（6类）、功能需求总览（含完整业务时序图、11个模块映射表）、非功能需求量化指标、**系统总体五层架构图**（含架构决策记录 ADR）
3 系统接口设计	三端页面跳转关系图、6个外部软件接口表、8对内部服务间接口、接口安全规范、数据结构逻辑设计（14个实体 + ER关系）、物理结构（索引策略11张表）、数据结构与程序关系映射
4 数据库设计	MySQL 主从架构方案、**10张核心表完整** DDL（含字段注释/索引）、Redis **11类 Key 设计**（类型/TTL/用途）
5 运行设计	容器部署拓扑图、7个服务端口规划与副本数、启动顺序、**下单支付完整时序（含异常路径）**、骑手位置上报控制、优惠券防超发控制、5类运行时段负载策略、数据容量预估
6 出错处理设计	统一错误码规范（6段）、10类异常场景处理策略、5类日志策略（含保留期）
7 注解	架构选型说明、WebSocket 消息链路说明、数据一致性保证、5条待细化设计点、需求→设计追踪矩阵

```
  ---
  Skills 与 MCP 服务使用说明

  Skills
```

Skill	调用时机	作用
se-architecture-doc-assistant	本次文档生成全程激活	提供概要设计工作流（架构决策记录、模块与接口拆分、数据设计、流程设计四步骤）、质量门禁（架构覆盖需求完整性、模块边界清晰度、接口可测试性）及强制约束（不编造组件、不与技术栈冲突）。文档结构与模板 assets/软件概要设计说明书.md 的章节体系完全对齐。

```
  MCP 服务（se-course-rag）
```

工具	实际调用情况	结果
search_course_docs	调用5次，查询"架构设计/模块划分/数据库/接口/运行部署"等主题	**返回 0 条**——RAG 向量集合当前为空，无历史概要设计类文档
get_phase_examples	未额外调用（search 已覆盖）	–
get_template_sections	通过 mcp_server.py 读取确认了模板章节结构	确认本文档章节与课程模板对齐

```
  RAG 检索结论：当前 Qdrant 集合中尚无已入库的历史概要设计文档（搜索返回 0 条），文档内容完全基于 SRS（软件需求规格说明书_饱了么.md）、项目3.md 需求描述及软件工程设计规范知识生成。

※ Cogitated for 9m 8s
```

两次查询向量数据库没有结果，转而直接根据skill目录下的报告模版和example示例文档生成软件概要说明书。本次总共用时9m8s，但是考虑到上次在没有查询数据库的情况花费5m多，其实也就大约是普通生成任务的两倍，而且这个api key调用起来本来就慢，真实如果换用其他反应快的模型速度肯定没这么慢。（过程中有报错，其实是本地使用python虚拟环境，因为没有激活，所以claude code运行脚本指令的时候出现了报错）

3.调用rag库生成回答的情况

```
❯ 根据已知文档（需求描述和之前生成的软件需求规格说明书），写一版软件详细设计说明书，如果有细节不清楚的地方，可以参考rag库中的知识（最后需要说明你使用了哪些skills和mcp服务）

● Read 2 files (ctrl+o to expand)

● 模板和概要设计都已读取。先并行查询 RAG，同时补充读取概要设计关键章节。

● Bash(cd "C:\Users\chuanzhenqi\Desktop\CS2026\AISE\rag_backend" && myenv/Scripts/python.exe -c "
      import sys, io, logging…)
  └ 安全测
    管理模块 非法访问、越权操作等 第
    试
    … +2 lines (ctrl+o to expand)

● RAG 有命中！提取完整内容。

● Bash(cd "C:\Users\chuanzhenqi\Desktop\CS2026\AISE\rag_backend" && myenv/Scripts/python.exe -c "
      import sys, io, logging…)
  └ Running in the background (↓ to manage)

● Read 1 file (ctrl+o to expand)

● Task Output bcbnny1kz
  └ 2026-03-08 22:48:49.811 | INFO     | vectordb.retriever:search:441 - ́́́́́́́query='́́́́́́́
```

├─ SS-07 商家管理	MQ消费接单流程、数据看板SQL设计
├─ SS-08 骑手配送	**智能派单评分算法（Python伪代码）**、抢单分布式锁流程、高德地图Adapter层降级
├─ SS-09 管理后台	审核+级联下架+操作日志完整流程
├─ SS-10 营销增值	**Redis DECR原子防超发算法**、周榜SQL生成逻辑
└─ SS-11 评价与通知	敏感词过滤、评分异步更新、推送失败指数退避重试
4 界面设计	统一响应/分页格式规范、完整28条接口清单（含认证/角色）、内部服务接口表、3个核心页面ASCII布局图
5 异常处理设计	10条错误码规范、全局异常处理器设计、8类故障预防与补救、优雅关机/滚动发布/备份策略
6 性能优化和安全设计	9项缓存优化策略（含收益量化）、限流/熔断/削峰设计、12类安全机制（SQL注入/XSS/CSRF/支付安全等）
7 项目测试计划	16条测试用例（单元/集成/系统/性能/安全/回归）+ 计划周次 + 测试工具规划

Skills 与 MCP 服务使用说明

Skills

Skill	作用
`se-architecture-doc-assistant`	提供详细设计工作流（模块与接口拆分、数据设计、流程设计四步法）及质量门禁约束（架构覆盖需求完整性、接口可测试性、不编造不存在组件）。文档结构与模板 `assets/软件详细设计说明书.md` 章节体系完全对齐。

MCP 服务（`se-course-rag`，工具：`search_course_docs`）

本次调用 RAG 的 Python 接口（`vectordb.retriever.search`）执行了 **8次主题检索，合计命中 144 条历史文档片段**，成功提取了以下有价值的参考内容：

检索主题	命中内容	实际影响
模块详细设计范式	往年"骑手配送模块"设计：`{模块编号、输入、处理、输出、算法设计}` 表格格式	采纳为本文所有子系统的属性表结构
骑手配送算法	往年："派单算法综合地理位置与时间窗口，使用 A* 路径规划算法优化路线"	参考并扩展为加权评分模型
接口设计规范	往年："RESTful 设计原则、HTTP 协议、JSON 格式、2xx/4xx/5xx 状态码、Apifox 管理"	强化了接口规范章节
安全设计	往年："全站 HTTPS、JWT、BCrypt、参数化查询防 SQL 注入、XSS 过滤"	在第6章安全设计中逐条对应并细化
性能优化	往年："热门商品缓存优化，图片 OSS 分发"	在第6章性能优化表中引用并扩展
异常处理	往年："订单提交失败自动回滚购物车；缓存容灾自动转为数据库读取"	在第5.2节故障补救表中直接采用
测试计划结构	往年：测试表格含"单元/集成/系统/性能/安全测试 + 第13周"的完整格式	直接复用表格结构，补充16条具体用例
界面设计	往年：主页"顶部搜索栏、中部推荐、底部菜单栏"的布局描述	在第4.3节用户界面设计中参考并细化

```
✻ Crunched for 10m 10s

● Background command "cd "C:\Users\chuanzhenqi\Desktop\CS2026\AISE\rag_backend" && myenv/Scripts/python.exe -c "
import sys, io, logging
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8')
logging.disable(logging.CRITICAL)
sys.path.insert(0, '.')
from vectordb.retriever import search

queries = [
    ('用户登录注册模块详细设计 接口参数 处理流程', 'design'),
    ('下单购物车模块详细设计 状态机 业务规则', 'design'),
    ('配送派单模块详细设计 算法 骑手调度', 'design'),
    ('接口详细设计 RESTful 请求响应格式 错误码', 'design'),
    ('安全设计 JWT认证 权限控制', 'design'),
    ('性能优化 Redis缓存 并发控制 限流', 'design'),
    ('测试计划 单元测试 集成测试用例', 'testing_deployment'),
    ('模块详细设计 输入输出 算法描述', 'design'),
]

for q, phase in queries:
    results = search(query=q, phase=phase, quality_level=['high'], use_reranker=True)
    if results:
        print(f'\n========== {q[:35]} ==========')
        for r in results[:2]:
            print(f'[score={r[\"composite_score\"]:.3f}] section={r[\"metadata\"].get(\"section_path\",\"\")} doc={r[\"metadata\"].get(\"doc_type\",\"\")}')
            print(r['text'][:600])
            print('---')
" 2>&1
" completed (exit code 0)

● 此后台任务的输出已在之前的 TaskOutput 调用中完整获取并处理，无需重复读取。

>
```

本次生成任务总共用时10m10s，相比之前没有查到rag信息的生成任务慢了一分钟左右，考虑到模型需要处理返回的查询结果和该任务的模版信息，时间比较合理。但感觉这接口调用得本来就有一点慢，如果模型调用速度快的话其实时间也就多了10%左右，是能接受的。毕竟如果假设不依赖rag数据库能1min输出，算上有rag搜索结果的时候也就最多2min